

Práctica Sección 1 + 2

Parte 1. (1 punto): Memoria de los ejercicios de clase.

1. **Memoria (PDF / Word):** Incluye una breve explicación de cada solución de los ejemplos de clase, capturas del terminal ejecutando el código y el enlace a tu repositorio de GitHub.
2. **Archivos de código:** Adjunta a la tarea los archivos .js de los ejercicios (individuales o en un .zip).

Checklist:

- [] Memoria adjunta con capturas y enlace a GitHub.
- [] Archivos .js subidos.

Parte 2. (9 puntos) Ejercicios Prácticos de JavaScript basados en Nose.js

Resuelve los siguientes ejercicios, añade capturas de pantalla y adjunta el archivo js final.

Ejercicio 1. El Grimoire Mutante del Mago



Conceptos: Declaración con 'const', Tipos por Referencia y Mutabilidad.

Misión: En la escuela de hechicería, un aprendiz ha declarado el libro de hechizos de la academia usando const:

```
//javascript
```

```
const grimoire = { owner: "Mago Merlín", spells: ["Bola de Fuego", "Escudo de Hielo"] };
```

Reto:

1. Agrega un nuevo hechizo ("Rayo Arcano") al array spells utilizando el método .push(). Explica en un comentario por qué Node.js permite esta modificación a pesar de que grimoire está definido con const.
2. Intenta reasignar grimoire = { owner: "Mago Oscuro" } y observa el error en consola. Escribe en un comentario qué tipo de error lanza Node.js y por qué sucede.

Ejercicio 2. Francotiradores Galácticos



Conceptos: Arrow Functions y Retorno Implícito.

Misión: Un videojuego espacial de consola requiere refactorizar el cálculo de daño de las naves a la sintaxis moderna de ES6+.

Reto:

1. Convierte las siguientes tres funciones tradicionales en Arrow Functions:

```
function calcularEscudo(nivel) { return nivel * 15; }
```

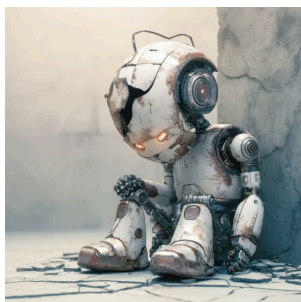
(Usa *sintaxis concisa de un solo parámetro sin paréntesis y retorno implícito*).

```
function impactoCritico(danoBase, multiplicador) { return danoBase * multiplicador; }
```

(Usa *retorno implícito de dos parámetros*).

```
function mensajeAlerta() { return "¡Alerta: Intrusos en la cubierta!"; }
```

(Usa *arrow function sin parámetros*).

Ejercicio 3. El Robot Mesero Desorientado

Conceptos: Métodos en Objetos y el problema de this con Arrow Functions.

Misión: En un restaurante ciberpunk, el robot cocinero CyberChef tiene un error de memoria y saluda diciendo: "Hola, soy undefined y preparo Ramen".

Código con Bug:

```
const robot = {
  name: 'CyberChef',
  specialty: 'Ramen',
  greet: () => {
    console.log(`Hola, soy ${this.name} y preparo ${this.specialty}`);
  }
};
robot.greet();
```

Reto:

1. Explica por qué `this.name` devuelve `undefined` al ejecutar la *arrow function* dentro del objeto.
2. Refactoriza el método `greet()` utilizando la **sintaxis recomendada para métodos ES6** (omitiendo la palabra clave `function` y los dos puntos) para que el robot recupere su identidad.

Ejercicio 4. Forjando Armas Legendarias

Conceptos: Transformación inmutable de Arrays con .map().

Misión: Un herrero de un RPG necesita encantar una lista de armas para venderlas en la tienda del reino.

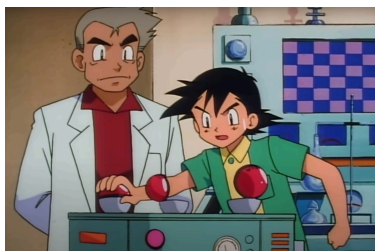
Datos de origen: `const armas = ["Espada de Hierro", "Hacha de Madera", "Daga de Cobre"];`

Reto:

1. Utiliza `.map()` con una arrow function para crear un nuevo array llamado `armasEncantadas` donde cada arma tenga el prefijo "Mística " y el sufijo " +1" (ej. "Mística Espada de Hierro +1").
2. Demuestra la inmutabilidad de `.map()` imprimiendo por consola el array original `armas` y el nuevo array `armasEncantadas` para comprobar que el original no ha cambiado.

Ejercicio 5.. Clonación Pokémon en el Laboratorio.

Conceptos: Operador Spread (...) en Objetos y Arrays.



Misión: El Profesor Oak quiere hacer un duplicado del expediente de un Pokémon para un experimento, agregándole nuevos atributos sin alterar los datos del Pokémon original.

Datos de origen:

```
const pokemonBase = { nombre: 'Pikachu', tipo: 'Eléctrico', nivel: 25 };  
const ataques = ['Impactrueno', 'Ataque Rápido'];
```

Reto:

1. Utiliza el **operador spread** para crear un nuevo objeto `pokemonMejorado` que copie todas las propiedades de `pokemonBase`, añada la propiedad variante: 'Shiny' y aumente el nivel a 50.
2. Crea un nuevo array `listaAtaques` usando *spread* que contenga los ataques originales más uno nuevo: 'Rayo'.

Ejercicio 6. La Receta del Barista Callejero

Conceptos: Operador Rest (...) en Funciones.



Misión: Estás programando el software de una máquina de café personalizada que acepta una base fija y cualquier cantidad de extra/ingredientes adicionales.

Reto:

1. Crea una arrow function llamada `prepararCafe` que reciba dos parámetros: el primero será tamaño (string, ej. "Grande") y el segundo será un parámetro Rest llamado `...ingredientes`.

2. La función debe retornar un mensaje resumido: "Café [tamano] con los siguientes extras: [lista de ingredientes separados por coma o array]".
3. Prueba llamar a la función con diferentes cantidades de ingredientes (ej. prepararCafe("Mediano", "Leche de Avena", "Vainilla", "Canela")).

Ejercicio 7. Hackeo de Datos del Servidor



Conceptos: Desestructuración de Objetos ({}) en parámetros de funciones.

Misión: Un sistema de ciberseguridad intercepta un objeto de respuesta de un servidor lleno de datos innecesarios, pero solo necesita tres campos para autorizar el acceso.

Datos interceptados:

```
const serverResponse = {
  status: 200,
  ip: '192.168.1.45',
  port: 8080,
  token: 'A98F72-ZX',
  sessionDuration: 3600,
  userRole: 'Admin'
};
```

Reto:

1. Crea una *arrow function* llamada validarAcceso que reciba directamente en sus parámetros las propiedades desestructuradas { ip, port, token }.
2. La función debe imprimir: "Conectando a IP: [ip] por el puerto [port] con el token [token]..."
3. Extrae también userRole mediante desestructuración directa en una línea de código independiente.

Ejercicio 8.El Podio de las Carreras Sci-Fi



Conceptos: Desestructuración de Arrays ([]) por posición.

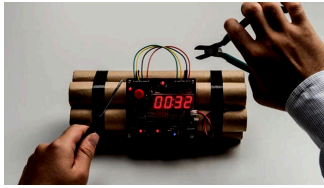
Misión: Al finalizar una carrera de naves voladoras, los resultados se guardan en un array ordenado según la llegada a la meta.

Datos: const posiciones = ["Nave Halcón", "Nave Rayo", "Nave Sombra", "Nave Cometa"];

Reto:

1. Utiliza la desestructuración de arrays en una sola línea para asignar el primer lugar a la constante oro, el segundo a plata y el tercero a bronce.
2. Imprime por consola: "Ganador de Oro: [oro], Plata: [plata], Bronce: [bronce]".

Ejercicio 9. El Desactivador de Bombas del Escape Room



Conceptos: Código Asíncrono, Creación de Promise y setTimeout.

Misión: Estás programando la lógica de un Escape Room virtual. El jugador introduce un código secreto para desactivar una bomba, y el sistema tarda 2 segundos en procesar el código.

Reto:

1. Crea una función `desactivarBomba(codigo)` que devuelva una nueva `Promise((resolve, reject) => { ... })`.
2. Dentro de la Promesa, usa un `setTimeout` de 2000 ms.
3. Si `codigo === "1234"`, llama a `resolve("¡Bomba desactivada con éxito! ")`.
4. Si el código es diferente, llama a `reject("¡Código incorrecto! Boooom")`.
5. Consume la función llamándola con un código y usando `.then()` para mostrar el mensaje de éxito y `.catch()` para manejar el error.